

ChaosJack

Giulia Bardi, Beatrice Pallotti, Elena Lungu, Evelyn Calandrini

Indice

1	Analisi	2
1.1	Descrizione e requisiti	2
1.2	Modello del Dominio	3
2	Design	5
2.1	Architettura	5
2.2	Design dettagliato	7
3	Sviluppo	19
3.1	Testing automatizzato	19
3.2	Note di sviluppo	20
4	Commenti finali	24
4.1	Autovalutazione e lavori futuri	24
4.2	Difficoltà incontrate e commenti per i docenti	26
A	Guida utente	27
B	Esercitazioni di laboratorio	29

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il software ChaosJack va a fondere il gioco classico di Black Jack con le meccaniche strategiche e i modificatori tipici di giochi come Balatro, un incontro tra il Poker classico e un videogioco di strategia e "rompicapo". L'obiettivo principale del giocatore è quello di sfidare e vincere sia contro il banco sia contro gli altri giocatori presenti al tavolo. Oltre ai giocatori effettivi saranno presenti dei giocatori bot, che seguiranno le regole di gioco in automatico. Svolgimento della partita:

- Fase iniziale: Per partecipare, ogni giocatore effettua una puntata base. Il banco distribuisce due carte scoperte a ciascun giocatore e a se stesso (scartando la prima e mostrando la seconda).
- Valore delle carte: Le figure valgono 10, l'Asso vale 1 o 11 a discrezione del giocatore, le altre carte mantengono il loro valore nominale.
- Turno del giocatore: In base al proprio punteggio, il giocatore può chiedere ulteriori carte ("hit") più volte, ma con un massimo di carte pescate. Se la somma supera 21, il giocatore sballa ed è fuori dalla partita. Il turno termina quando il giocatore decide di fermarsi ("stand").
- Fase di puntata finale : Terminato il turno di tutti i giocatori, è possibile effettuare ulteriori scommesse scegliendo una somma o raddoppiando il precedente (questo solo se si posseggono due carte in mano).
- Turno del banco: Il banco pesca carte fermandosi obbligatoriamente al raggiungimento di un punteggio pari o superiori a 17. Se il banco supera 21, la vincita è contesa tra i giocatori ancora attivi con il punteggio più alto.

Condizioni di vittoria e punteggi : Essendo un tutti contro tutti, per vincere bisogna battere sia il banco che gli altri giocatori. A parità di punteggio, vince chi ha meno carte in mano. Se il pareggio persiste tra più giocatori, la vincita viene divisa. La vincita standard raddoppia il piatto. Il raggiungimento del Blackjack triplica il piatto, mentre una mano monocolora fornisce un moltiplicatore bonus. Inoltre, all'avvio di una partita, il giocatore potrebbe imbattersi in turni "speciali" e carte speciali con vincoli che alterano i punteggi.

Requisiti funzionali

- Il sistema deve permettere lo svolgimento della logica di base del Blackjack, includendo la distribuzione iniziale delle carte e il loro mescolamento. Inoltre deve garantire le azioni di richiesta delle carte, fermarsi, uscire e puntare.
- Il sistema deve calcolare correttamente i punteggi delle mani, gestendo la soglia massima del 21 e le regole specifiche associate alle singole carte.
- Il gioco deve applicare automaticamente le alterazioni delle regole durante i turni speciali.
- Il software deve gestire i portafogli virtuali (fish) sia del giocatore che del banco, aggiornandoli in base all'esito delle puntate.

Requisiti non funzionali

- L'interfaccia utente deve essere reattiva, restituendo un feedback visivo immediato a ogni azione del giocatore senza ritardi percettibili.
- Il sistema deve supportare l'aggiunta futura di nuovi modificatori, turni speciali o regole.

1.2 Modello del Dominio

L'ambiente principale è il tavolo, il quale riunisce tutte le entità coinvolte e scandisce lo svolgimento del gioco. Al tavolo siedono i partecipanti, ovvero il banco, i giocatori e i giocatori bot (i quali seguono logiche autonome). Ciascun partecipante è dotato di un proprio portafoglio di fiches e riceve una mano di carte. L'intero flusso della partita al tavolo è organizzato in una sequenza di fasi ben distinte (come la puntata iniziale o la fase di gioco). All'interno di ogni fase, l'azione si articola nei turni individuali dei partecipanti.

Durante le fasi di scommessa, le fiches convergono in un piatto centrale. Le carte vengono fornite da un mazzo, che include sia le carte classiche che speciali. Infine, specifiche partite possono essere alterate dall'applicazione di modificatori, i quali introducono vincoli temporanei alle regole di base alterando le normali fasi di gioco.

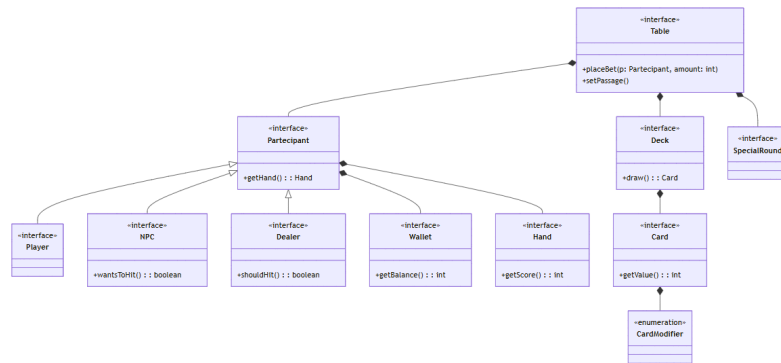


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2

Design

2.1 Architettura

L'architettura adottata per il progetto ChaosJack segue il pattern architetturale MVC(Model-View-Controller). Questo pattern consente di avere una chiara separazione delle responsabilità e una maggiore flessibilità in fase di sviluppo, permettendoci di separare la logica di gioco dalla sua rappresentazione grafica. Ovvero è possibile sostituire il blocco della view senza richiedere alcuna modifica nel Controller e nel Model. L'architettura si basa su queste componenti principali:

- Model : GameEngine e Table
- Controller : GameFlowController e ActionController
- View : ViewManager

Il Model : Si occupa di gestire e mantenere in modo consistente lo stato interno del gioco (il mazzo di carte, i portafogli dei partecipanti, il calcolo dei punteggi nelle mani). Valida ed esegue tutte le mutazioni dello stato di gioco secondo le regole tradizionali del BlackJack e l'applicazione dei relativi modificatori speciali.

Il controller : Rappresenta il motore attivo che si occupa di dettare i tempi e l'ordine delle diverse fasi della partita (regolando i turni tra i giocatori presenti nel tavolo).

La View: Ha il ruolo generale di gestire la visualizzazione dei componenti all'interno della finestra, mostrando l'evoluzione della partita. Si occupa, inoltre, di gestire la fase di navigazione del menù, consentendo di avviare una nuova partita o vedere le statistiche dei round giocati. P

Grazie a questa architettura è possibile sostituire il blocco della view senza richiedere alcuna modifica nel Controller e nel Model. La View si limita a notificare le azioni dell'utente al Controller. Sarà poi il Controller a invocare i metodi appropriati sul Model per aggiornare lo stato.

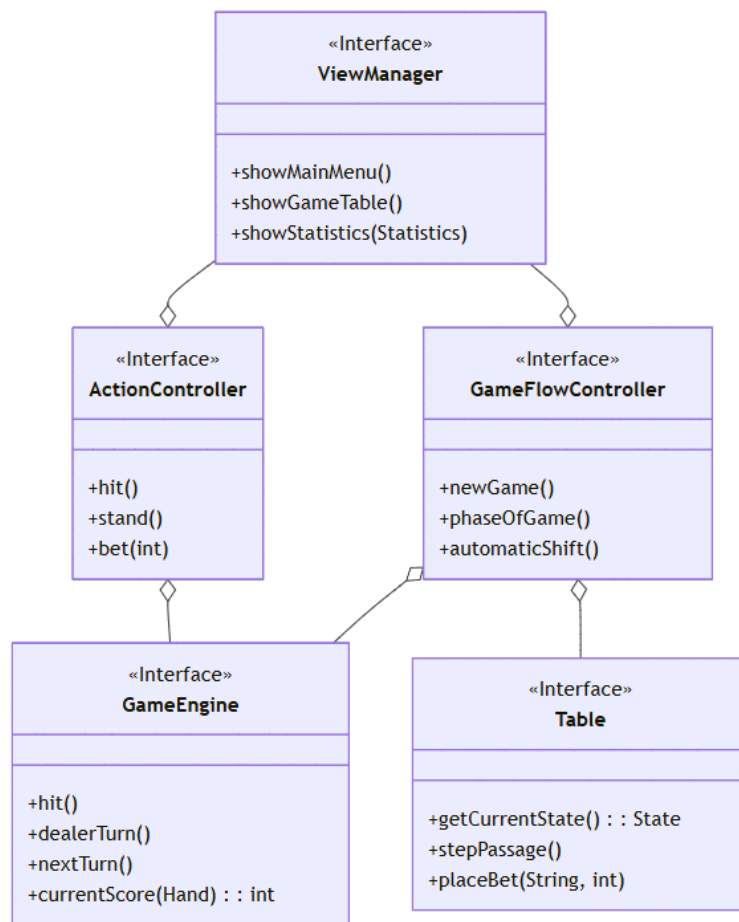


Figura 2.1: Schema UML dell'architettura MVC realizzata per ChaosJack

2.2 Design dettagliato

Evelyn Calandrini - Gestione del Tavolo, Valutazione dei Round, Statistiche, View

Il contributo si è focalizzato sulla definizione del tavolo che si occupa di riunire il gioco (partecipanti, carte e regole di gioco) occupandosi poi alla fine del round di valutare il vincitore della partita. Si aggiornano così le statistiche salvando il risultato della partita. Inoltre il contributo va nell'interfaccia con la gestione del cambio di finestra dalla situazione di gioco, menu, pausa e statistiche.

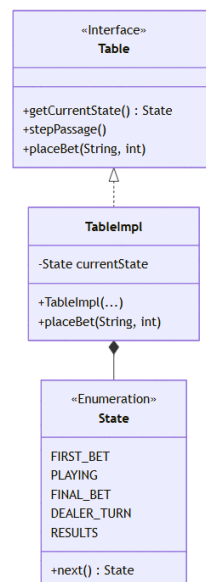


Figura 2.2: Schema UML rappresenta la gestione delle fasi di gioco del tavolo

Gestione dello Stato e delle puntate

Problema : In un ambiente di gioco a turni, è necessario garantire che le azioni dei singoli giocatori (come effettuare una puntata o chiedere le carte) avvengano esclusivamente nelle fasi corrette di gioco, prevenendo stati inconsistenti o alterazioni illecite del piatto.

Soluzione : L'alternativa di usare variabili boolean è stata scarta perchè incline ad errori. Il sistema utilizza un approccio basato su stati di gioco differenziati per le varie fasi di gioco. Il tavolo (TableImpl) verifica lo stato corrente prima di ogni operazione. In questo modo le varie azione di gioco controllano prima di trovarsi nello stato corretto, se l'azione non è ammessa l'esecuzione viene interrotta.

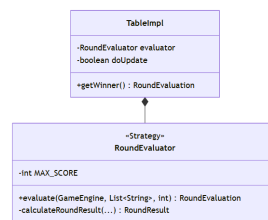


Figura 2.3: Schema UML rappresenta l'algoritmo di valutazione (Pattern Strategy)

Valutazione dei risultati

Problema : La decretazione del vincitore nel BalckJack richiede la comparazione simultanea dei punteggi di tutti i giocatori contro il banco, tenendo conto delle regole che lo differiscono dal gioco tradizionale (mani monocolori, situazioni di pareggio e turni speciali). Il problema è fare in modo che il tavolo sia all'oscuro di come sia il calcolo per decretare il vincitore in caso ci sia un cambio di regole futuro.

Soluzione : L'alternativa di implementare il calcolo direttamente all'interno della classe del tavolo è stata scartata, perchè l'avrebbe resa troppo complessa. Si prevede l'isolamento dell'algoritmo di calcolo in una classe esterna RoundEvaluator. Questa classe si occupa di processare lo stato del GameEngine e restituisce l'esito formattato all'interno del record RoundEvaluation. In questo modo il tavolo si focalizza sul coordinamento della partita. Facendo così però si ha la necessità di iniettare riferimenti esterni, come il GameEngine, e il piatto corrente al valutatore ad ogni fine turno. La soluzione adotta il pattern Strategy. Il tavolo da gioco delega la strategia per la valutazione del round alla classe RoundEvaluator.

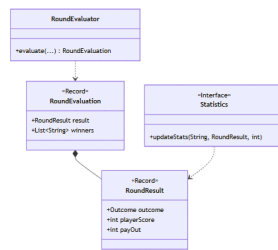


Figura 2.4: Schema UML rappresenta le statistiche e il trasferimento dei dati

Statistiche e Trasferimento Dati

Problema : Al termine di ogni round, è necessario comunicare l'esito della partita (vincitori, payout, punteggi) verso il modulo delle statistiche e verso la View. Il rischio principale era l'esposizione di riferimenti diretti agli oggetti del Model, che avrebbe permesso modifiche accidentali allo stato del gioco dall'esterno.

Soluzione : Un'alternativa sarebbe stata l'interrogazione diretta dei metodi getter del tavolo da parte dei moduli esterni. Questa soluzione è stata scartata poiché avrebbe creato un forte accoppiamento e la possibilità di manipolare indebitamente i dati del Model. La soluzione adottata utilizza i Record. I record **RoundResult** e **RoundEvaluation** sono intrinsecamente immutabili. Questo garantisce che, una volta generato l'esito della partita, il pacchetto di dati sia protetto da qualsiasi alterazione, anche se comporta l'allocazione di nuove istanze a ogni round. La soluzione utilizza il pattern Data Transfer Object (DTO). I record agiscono come contenitori passivi, trasportando informazioni dal Model verso i restanti componenti del sistema in modo isolato e non modificabile, eliminando qualsiasi dipendenza logica tra chi produce il dato e chi lo consuma.

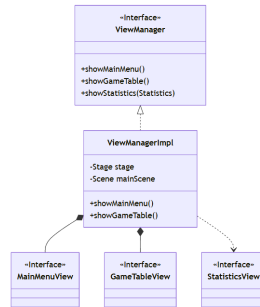


Figura 2.5: Schema UML che rappresenta il ViewManager (Pattern Facade)

Gestione dell'Interfaccia (ViewManager)

Problema : Si prevedono diverse schermate grafiche (Menu, Tavolo di Gioco, Statistiche, Pausa). Gestire la transizione tra le scene e la pulizia delle finestre distribuendo la logica nei vari Controller avrebbe accoppiato irrimediabilmente la logica di gioco alla libreria JavaFX.

Soluzione : Come soluzione si andrà ad utilizzare un gestore centralizzato, il ViewManager. Invece di effettuare chiamate incrociate tra le varie schermate, il Controller interagisce esclusivamente con questa interfaccia unificata. Il ViewManager si occupa internamente di scambiare i nodi e aggiornare lo Stage principale. Questa soluzione reifica il pattern Facade, fornendo un punto di accesso semplificato e isolando i Controller dalle complessità della libreria grafica.

Elena Lungu - Gestione delle carte speciali, dei modifcatori, wallet e view

In questa sezione verranno analizzate le scelte progettuali relative alla modellazione delle carte da gioco e del portafoglio del giocatore, componenti fondamentali per l'esperienza di gioco di ChaosJack.

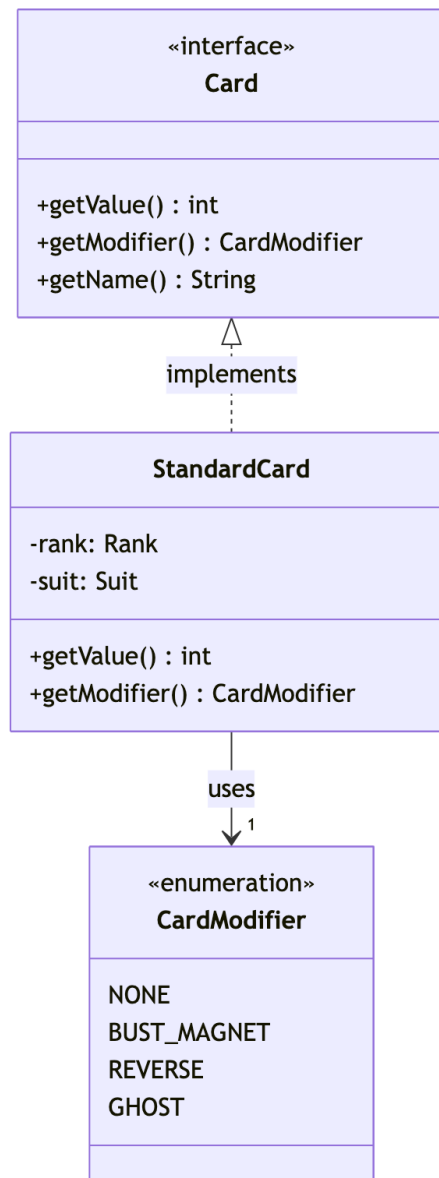


Figura 2.6: Diagramma UML che illustra la relazione tra **StandardCard** e **CardModifier**.

Gestione delle Carte Speciali e Modificatori

Problema : Il gioco ChaosJack introduce la presenza di carte speciali (dette carte "Chaos") che alterano le regole standard del Blackjack, modificando i punteggi o il normale flusso di gioco attraverso l'uso di carte Bust, Reverse, Ghost. Creare una sottoclasse di **StandardCard** per ogni tipo di carta

speciale avrebbe portato a una gerarchia rigida, difficile da mantenere e soggetta a esplosione combinatoria, rendendo complesso aggiungere nuovi effetti in futuro.

Soluzione : Al fine di modellare le carte speciali, si è scelto di applicare il principio fondamentale *"Prefer Composition over Inheritance"*, evitando la creazione di una complessa gerarchia di sottoclassi. È stato introdotto il tipo **CardModifier**, il quale incapsula la logica necessaria ad alterare il valore della carta. In pratica, una **StandardCard** riceve il proprio modificatore al momento della creazione; da quel momento in poi, ogni volta che viene richiesto il punteggio della carta, quest'ultima si limita a delegare il calcolo direttamente al suo modificatore.

Questa scelta progettuale è risultata nettamente superiore all'uso dell'ereditarietà per tre motivi principali:

1. **Evita la Class Explosion:** L'uso dell'ereditarietà avrebbe richiesto la creazione di una sottoclasse per ogni singolo effetto (es. **BustCard**, **ReverseCard**, **GhostCard**). In presenza di numerose carte "Chaos", il numero di classi del dominio sarebbe diventato altissimo, rendendo la base di codice difficile da gestire.
2. **Rispetta il Principio Open/Closed:** Il sistema è aperto all'estensione ma chiuso alla modifica. Nuovi modificatori possono essere introdotti semplicemente aggiungendo elementi all'enumerazione, lasciando totalmente invariata la logica interna della classe **StandardCard**.
3. **Garantisce flessibilità per future combinazioni:** L'ereditarietà in Java è singola e statica, il che renderebbe impossibile applicare più effetti contemporaneamente senza ricorrere a classi duplicate. Con la composizione, l'architettura è già predisposta per evolvere verso una **List<CardModifier>** all'interno di **StandardCard**, permettendo di iterare e applicare combinazioni illimitate di effetti con estrema scalabilità.

La soluzione messa in campo rappresenta un'applicazione del pattern *Strategy*. Invece di definire il comportamento del calcolo del punteggio staticamente all'interno della classe, la **StandardCard** (che agisce come *Context*) delega il calcolo del suo valore finale al **CardModifier** che le è stato iniettato.

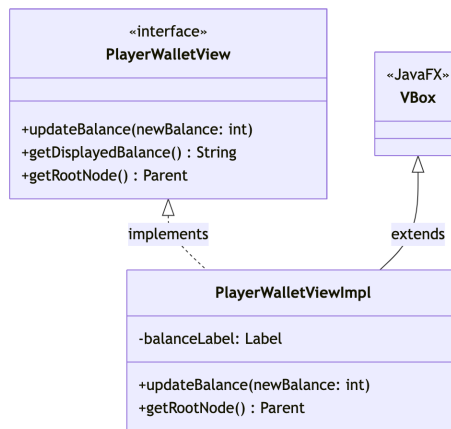


Figura 2.7: Diagramma UML che mostra l'incapsulamento della logica grafica in `PlayerWalletViewImpl`.

Disaccoppiamento e Rappresentazione del Wallet (View)

Problema : Durante la partita, il giocatore deve avere costantemente sotto controllo il proprio saldo (fiches). Era necessario creare un'interfaccia utente grafica per il portafoglio che fosse indipendente dalla logica di gestione dei dati (`StandardWallet`), facilmente aggiornabile in tempo reale, e che garantisse uno stile visivo accattivante senza alterare le altre componenti della GUI.

Soluzione : È stata creata un'interfaccia specifica `PlayerWalletView` e la sua implementazione `PlayerWalletViewImpl`, che estende direttamente un componente JavaFX. L'implementazione maschera totalmente i dettagli grafici (come font, bordi colorati e l'effetto `DropShadow`) esponendo unicamente metodi di alto livello come `updateBalance(int newBalance)`.

Come alternativa si era inizialmente valutato di inserire la struttura grafica del portafoglio direttamente nel file FXML principale della partita. Tuttavia, si è optato per l'estrazione in un componente JavaFX dedicato per massimizzare la modularità, il riuso e la *Separation of Concerns*. Qualora in futuro il gioco venisse esteso per supportare più giocatori a schermo (es. multiplayer locale), l'attuale design permetterebbe di istanziare molteplici `PlayerWalletViewImpl` senza alcuna duplicazione di codice. Inoltre, questa scelta mantiene il Controller principale disaccoppiato e pulito, poiché quest'ultimo interagisce con la View del Wallet solo tramite metodi ad alto livello, senza conoscere i dettagli del rendering interno o la gestione dei nodi figli.

Qui emerge chiaramente l'uso architetturale del pattern MVC applicato ai singoli componenti. `PlayerWalletViewImpl` agisce come View pura: non possiede alcuna logica di gestione dei dati, non calcola le puntate né verifica se il saldo sia sufficiente (compiti di competenza del Model `StandardWallet`), ma si limita a visualizzare il valore fornito dal Controller tramite il metodo di aggiornamento.

Giulia Bardi-Gestione dei partecipanti, protezione input e distinzione Uman-bot

In questa sezione verranno analizzate le scelte progettuali relative alla modellazione della gerarchia dei giocatori e del controller delle azioni del gioco.

Gerarchia dei Partecipanti(Giocatori,Dealer e bot)

Problema Nella struttura di ChaosJack sono presenti diverse tipologie di giocatori che prendono parte al gioco:Il giocatore umano(`Player`), il bot(`NPC`) e il banco (`Dealer`). Ognuno di essi possiede logiche diverse ma tutti e tre condividono delle caratteristiche come il nome, una mano di carte e nel caso del giocatore umano e del bot anche un portafoglio. Implementare separatamente queste tipologie avrebbe portato ad una grande quantità di duplicazione del codice e difficoltà di gestione per il sistema.

Soluzione Per risolvere questo problema si è scelto di creare un'interfaccia alla base `Participant` da cui poi derivano delle interfacce più specifiche per le varie tipologie di partecipanti come `Dealer`, `Player` e `NPC`. Per evitare la duplicazione del codice nell'implementazione delle interfacce è stata creata la classe astratta `AbstractPlayer` che fornisce l'implementazione di base per le classi condivise. L'Ereditarietà delle classi:

1. `DealerImpl` estende `AbstractPlayer` dove si implementano solo le logiche esclusive del banco.
2. `PlayerImpl` estende `AbstractPlayer` e aggiunge la gestione del portafoglio e delle fisch `Wallet`.
3. `NPCimpl` estende `Playerimpl` ereditando tutta la gestione economica aggiungendo solo la logica esclusiva ai bot.

Questa architettura utilizza il Polimorfismo e il principio della Composizione.

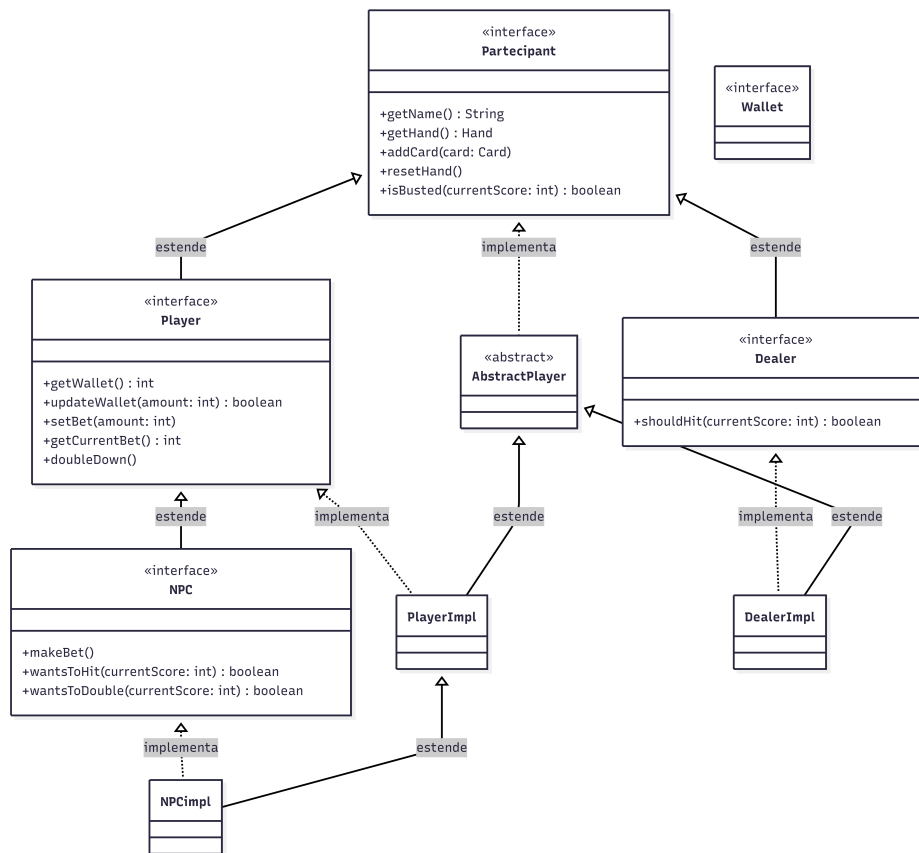


Figura 2.8: Diagramma UML delle classi relative ai Partecipanti

Protezione degli input e Distinzione Umano-bot

Problema Essendo un gioco a turni dove si mischiano giocatori umani e giocatori automatizzati come i bot e il banco si crea un problema di consistenza degli input. L'interfaccia grafica invoca i metodi pubblici del controller come `hit()`, `stand()` e `doubleDown()` quando l'utente preme il pulsante. Se l'utente preme tali pulsanti durante il turno di un bot ci sarebbe il rischio di alterare lo stato del gioco.

Soluzione I metodi di azioni dell'`ActionControllerImpl` sono diventati più controllati. Si ha implementato un metodo privato `getCurrentHumanPlayer()`, il quale verifica se il partecipante è effettivamente un giocatore umano oppure è un bot. Nel caso in cui si tratti di un bot, non è permesso all'utente premere i vari pulsanti.

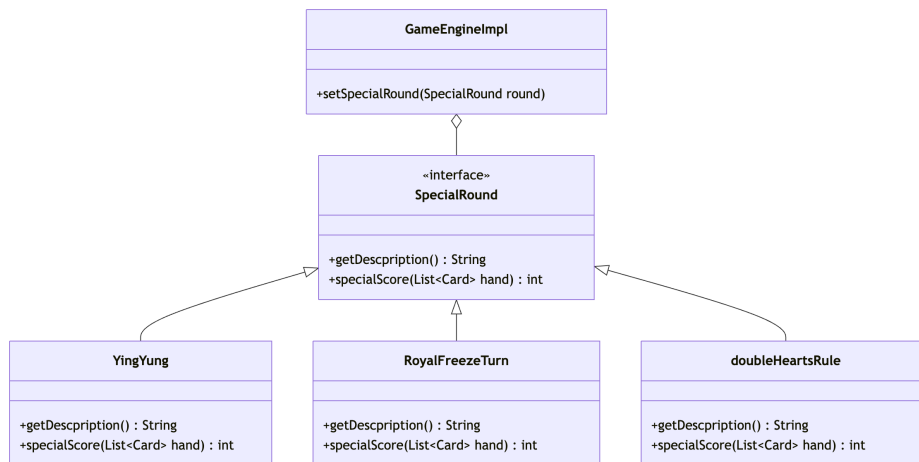


Figura 2.10: diagramma delle classi special round

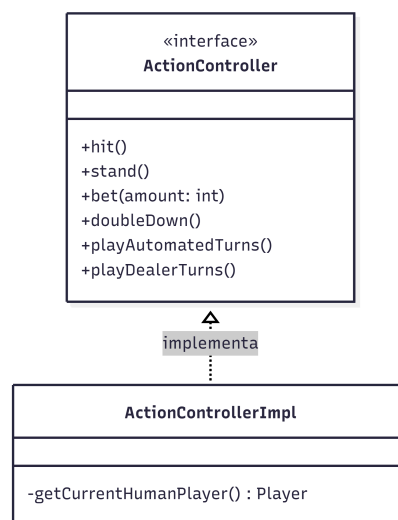


Figura 2.9: Diagramma UML per l'ActionController

Beatrice Pallotti- Gestione dei round speciali, gestione dei turni e del flusso di gioco

In questa sezione si discuterà delle scelte progettuali per quanto riguarda la gestione dei round speciali, del flusso del gioco e della gestione dei turni.

Creazione e gestione dei round speciali

problema: Il gioco ChaosJack prevede la presenza di più round speciali, ciascuno caratterizzato da delle precise regole per il calcolo dei punteggi delle mani dei giocatori. La dinamica del gioco prevede che questi round speciali si attivino in maniera casuale all'inizio dei singoli round del gioco. La gestione di tutti questi round, con la relativa logica di calcolo degli score in una sola classe avrebbe portato ad un ampio uso di if-else rendendo la classe poco leggibile e poco flessibile a possibili cambiamenti.

Soluzione: Per evitare il problema spiegato sopra si è optato per l'utilizzo del pattern di tipo Strategy che mi permette di andare a gestire in maniera dinamica l'attivazione oppure no dei round speciali all'interno del gioco.

1. L'interfaccia `SpecialRound` è la **Strategy Interface** contenente il metodo `specialScore(List<Card> playerscards)`.
2. Le **Concrete Strategies** sono le tre classi che implementano in maniera concreta i round speciali, esse sono: `DoubleheartsRule`, `YingYung` e `RoyalFreezeTurn`. Ognuna di essa implementa le proprie regole per il calcolo del punteggio.
3. Il **Context** è la classe `GameEngineImpl` la quale ha `Optional<SpecialRound>` come riferimento al round attivo in quel momento del gioco (null se il round attivo utilizza le regole base del gioco). Tramite il metodo `currentScore()` viene gestito il calcolo del punteggio richiamando le regole del round attivo.

Con questa soluzione si ha la possibilità di andare ad aggiungere in futuro eventuali round speciali creando una classe che vada ad implementare `SpecialRound`

Model e controller

Problema: per determinate azioni del gioco è richiesta la coordinazione e il richiamo di più classi del dominio, un esempio può essere la gestione dei turni dei giocatori, la logica che permetta al giocatore di poter passare il proprio turno in maniera volontaria e , come detto prima, la logica con cui selezionare e implementare la regola per il calcolo dei punteggi. Se dovessimo delegare il controller alla gestione di tutte queste logiche il codice diventerebbe troppo pesante dato che si dovrebbe già occupare della gestione del flusso di gioco.

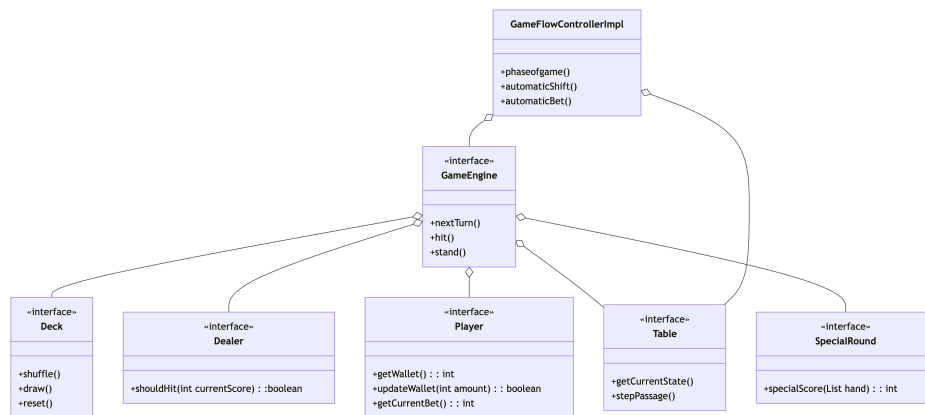


Figura 2.11: diagramma relativo al Gameengine e il GameFlowController

Soluzione: Per risolvere il problema è stata creata la classe **GameEngineImpl** al cui interno si trovano i metodi che servono a gestire le logiche più complesse trattate sopra come ad esempio **nextTurn()** (per i turni dei giocatori), **stand** (per permettere ai giocatori di passare il turno volontariamente), **currentScore** (per applicare le giuste regole di calcolo del gioco), **resetGame** (che permette di andare a inizializzare il gioco ogni volta che si inizia un round). Con questa soluzione il controller potrà utilizzare queste logiche all'interno del suo codice andando a richiamare dei semplici metodi.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il processo di testing automatizzato del progetto è stato realizzato per assicurare il corretto funzionamento del codice nel tempo, attraverso una suite di test automatici JUnit. La strategia di verifica si è concentrata esclusivamente sul model e sui controllori di flusso. Si è scelto di non testare la view e il controller e di concentrarsi a fare i soli test del model per garantire un corretto funzionamento del software. La suite di test è stata suddivisa nelle seguenti aree funzionali:

- **Test del motore di gioco e tavolo** (`GameEngineImplTest`, `TableTest`):
I test garantiscono che il ciclo di vita della partita sia corretto, che le transizioni di stato (es. da puntata a gioco) siano valide e che le regole di gioco vengano applicate rigorosamente. Sono stati inclusi test sui "casi critici", basati sulle regole speciali, per verificare che il tavolo risponda correttamente ad input non previsti.
- **Test delle Entità e Logica di Dominio** (`DealerTest`, `PlayerTest`, `HandTest`, `WalletTest`): Questi test verificano l'integrità delle componenti base. Si è garantito che la somma dei punteggi delle mani (`Hand`) sia calcolata correttamente (gestendo il valore flessibile dell'Asso, carte speciali e round speciali) e che il portafoglio (`Wallet`) impedisca transazioni non coperte dal saldo. È stata validata la correttezza del flusso di denaro: il saldo del portafoglio decresce correttamente all'atto della scommessa e viene incrementato in caso di vincita, assicurando che non si verifichino mai saldi negativi o creazione illecita di credito.
- **Test della Logica di Carte e Mazzi** (`StandardCardTest`, `StandardDeckTest`): i test verificano che il mazzo contenga sempre il numero corretto di

carte e che la pesca sia casuale e senza duplicati, prevenendo anomalie algoritmiche nella distribuzione.

- **Test delle Statistiche (StatisticsTest):** I test simulano una serie di round con esiti differenti e validano che il contatore finale di vittorie/sconfitte e il bilancio economico siano esatti, garantendo l'assenza di regressioni nei calcoli storici.

3.2 Note di sviluppo

Evelyn Calandrini

- **Utilizzo della libreria JavaFX per l'interfaccia grafica** Utilizzata per la gestione strutturale e il rendering del menu principale, del tavolo da gioco e della schermata delle statistiche. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/view/impl/ViewManagerImpl.java#L31-L32>
- **Utilizzo di lambda expression per i listener di eventi sulle properties** Usate per implementare in modo compatto i listener di ridimensionamento della scena (`widthProperty` e `heightProperty`). Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/view/impl/ViewManagerImpl.java#L47-L48>
- **Utilizzo di Stream API e costrutti funzionali (uso concatenato di filter, map, findFirst e ifPresent con Optional)** Utilizzati per l'ispezione sicura della lista dei giocatori del motore di gioco e per l'aggiornamento del saldo dei rispettivi wallet. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/TableImpl.java#L107-L111>
- **Utilizzo del costrutto avanzato Switch Expressions** Utilizzato all'interno della gestione delle statistiche per smistare e aggiornare i contatori storici in base all'esito del round (`Outcome`). Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/StatisticsImpl.java#L24-L44>
- **Utilizzo di riferimenti a metodi (Method Reference) e interfacce funzionali sulle Map** Utilizzati in combinazione con il me-

`todo Map.merge()` e `Integer::sum` per gestire e incrementare i piatti delle puntate in modo atomico. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/TableImpl.java#L87>

- **Utilizzo di annotazioni di librerie di terze parti per l'analisi statica (SpotBugs / FindBugs)** Utilizzata l'annotazione `@SuppressWarnings` sul costruttore del tavolo per documentare l'esposizione intenzionale del riferimento al `GameEngine`. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/TableImpl.java#L37-L40>

Giulia Bardi

- **Utilizzo della classe `Optional` e costrutti funzionali**
Utilizzato per il recupero del giocatore umano all'interno di `ActionController`. Permalink : <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/61dbd5ecbfb5b63ae1a78c85f03cce47da244ca/src/main/java/it/unibo/chaosjack/controller/impl/ActionControllerImpl.java#L185>
- **Utilizzo di `lambda expressions`**
Utilizzati nel controller tramite `map` per il casting a run-time. Permalink : <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/61dbd5ecbfb5b63ae1a78c85f03cce47da244ca/src/main/java/it/unibo/chaosjack/controller/impl/ActionControllerImpl.java#L186>
- **Utilizzo di riferimenti a metodi (Method Reference)**
All'interno dell'`Optional` del Controller si è sfruttato il costrutto `this::isHumanPlayer` per passare la firma del metodo. Permalink : <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/61dbd5ecbfb5b63ae1a78c85f03cce47da244ca/src/main/java/it/unibo/chaosjack/controller/impl/ActionControllerImpl.java#L187>
- **Utilizzo di annotazioni di librerie di terze parti (SpotBugs)**
L'annotazione `@SuppressWarnings` è stata utilizzata per gestire l'esposizione intenzionale dei riferimenti interni. Permalink : <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/61dbd5ecbfb5b63ae1a78c85f03cce47da244ca/src/main/java/it/unibo/chaosjack/controller/impl/ActionControllerImpl.java#L31> <https://github.com/evelyncalandrini/OOP25-chaosjack/>

blob/61dbd5ecbfbb5b63ae1a78c85f03cce47da244ca/src/main/java/
it/unibo/chaosjack/model/impl/AbstractPlayer.java#L39

Beatrice Pallotti

- **Utilizzo della classe Optional** Utilizzata per il riferimento al round speciale. Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/it/unibo/chaosjack/model/impl/GameEngineImpl.java#L28>
- **Utilizzo di stream** Utilizzato per controllare che tutti i giocatori (ad eccezione del dealer) abbiano sballato (abbiano uno score maggiore di 21). Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/it/unibo/chaosjack/controller/impl/GameFlowControllerImpl.java#L236>
- **Utilizzo delle lambda** Utilizzate nel collegamento con i bottoni della view Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/it/unibo/chaosjack/controller/impl/GameFlowControllerImpl.java#L114>
- **Utilizzo libreria esterna JavaFX** Ho utilizzato questa libreria per implementare il PauseTransition. Esso è stato utilizzato per far sì che ci sia una pausa temporale tra le azioni dei bot (npc e dealer). Utilizzo questa libreria per utilizzare Platform.exit() per chiudere l'interfaccia grafica. Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/it/unibo/chaosjack/controller/impl/GameFlowControllerImpl.java#L302> Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/it/unibo/chaosjack/controller/impl/GameFlowControllerImpl.java#L86>
- **Utilizzo annotazioni di librerie di terze parti (SpotBugs)** @SuppressWarnings è stata utilizzata per gestione dell'esposizione intenzionale a riferimenti esterni Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/it/unibo/chaosjack/controller/impl/GameFlowControllerImpl.java#L53> Permalink:<https://github.com/evelyncalandrini/00P25-chaosjack/>

blob/a372dd05d690b559e4186b1993c1903b468c322d/src/main/java/
it/unibo/chaosjack/model/impl/GameEngineImpl.java#L115

l'esempio di utilizzo per il `pauseTransition` è stato preso dal seguente link:
<https://stackoverflow.com/questions/27334455/how-to-close-a-stage-after-a-certain-time>

Elena Lungu

- **Utilizzo della classe `Optional`** Utilizzata all'interno della classe `StandardDeck` per gestire in modo sicuro la pesca di una carta dal mazzo, prevenendo la restituzione di valori nulli qualora il mazzo fosse vuoto. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/StandardDeck.java#L47-L51>
- **Utilizzo della libreria `JavaFX` e costrutti `SVGPath`** Oltre all'impiego per la struttura generale delle viste del portafoglio e delle singole carte (`CardViewImpl` e `PlayerWalletViewImpl`), la libreria è stata sfruttata a un livello più avanzato tramite `javafx.scene.shape.SVGPath` per disegnare programmaticamente in formato vettoriale i simboli dei modificatori sulle carte. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/view/impl/CardViewImpl.java#L164-L188>
- **Utilizzo della classe di utilità `Collections`** Impiegata in combinazione con la classe `StandardDeck` per applicare algoritmi di randomizzazione standard (`Collections.shuffle`) sia al mazzo di carte principale che al pool di modificatori speciali, garantendo un'adeguata casualità distributiva. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/StandardDeck.java#L60-L61>
- **Utilizzo estensivo di `Enum` per il Domain Driven Design** Sfruttamento avanzato delle enumerazioni (es. `Suit`, `Rank`, `CardModifier`) per modellare in modo robusto le caratteristiche immutabili delle carte da gioco senza l'ausilio di magic strings o magic numbers. Permalink: <https://github.com/evelyncalandrini/OOP25-chaosjack/blob/master/src/main/java/it/unibo/chaosjack/model/impl/StandardCard.java>

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

Evelyn Calandrini

Il mio contributo al progetto si è focalizzato sulla gestione del tavolo di gioco, sul sistema delle statistiche e sull'interfaccia utente. La gestione del tavolo ha rappresentato la sfida tecnica più complessa, essendo il nucleo centrale per il coordinamento delle varie componenti del gioco. Parallelamente, ho sviluppato un sistema per raccogliere le statistiche in modo chiaro e progettato un'interfaccia che risultasse il più semplice e intuitiva possibile. Ritengo il lavoro complessivo molto soddisfacente, con un'ottima integrazione delle mie funzionalità nel resto del progetto. Questa esperienza è stata particolarmente formativa perché ha evidenziato sul campo l'importanza di una progettazione modulare, fondamentale per facilitare la collaborazione in team e per poter estendere il codice senza stravolgerlo. In un'ottica di sviluppo futuro, il progetto offre ampi margini di miglioramento. Il sistema delle statistiche potrebbe essere ampliato integrando lo storico delle partite e le classifiche, mentre l'interfaccia grafica trarrebbe beneficio da una maggiore cura estetica tramite l'aggiunta di animazioni e feedback visivi. Infine, la gestione del tavolo potrebbe sfruttare l'attuale modularità per supportare facilmente nuove modalità di gioco.

Giulia Bardi

Il mio contributo nel progetto è stata nella gestione delle varie tipologie di partecipanti e della gestione delle azioni principali che posso eseguire i vari partecipanti. La parte più impegnativa è stata la gestione dei partecipanti principalmente per la scelta di come implementare il polimorfismo per evitare

la duplicazione del codice. Invece per il controller è stato insidioso dovendo fungere da mediatore per proteggere l'integrità del Model durante i turni. Ritengo il lavoro soddisfacente con un'architettura che ha raggiunto un alto grado di disaccoppiamento. Questa esperienza è stata particolarmente formativa per la comprensione profonda dei pattern architetturali sul campo. Nello specifico, dover implementare l'MVC mi ha fatto scontrare con la necessità di separare nettamente le responsabilità. Inoltre mi ha permesso di lavorare in team e dividersi le responsabilità del lavoro. In un'ottica di sviluppo futuro, il progetto offre spunti di espansione come il miglioramento della logica intelligente per il bot, rendendolo più imprevedibile per il giocatore.

Beatrice Pallotti

il mio contributo per il progetto è stato nella gestione dei round speciali, nella gestione dei turni, nel creare la mano del giocatore e il controllo di flusso del gioco. Le parti più impegnative sono state su come controllare il flusso dovendo tener conto delle regole stabilite per il gioco e il coordinamento di tutte le classi del dominio dovendo utilizzarle tutte insieme per implementare determinate logiche. Questa esperienza è stata molto formativa dal punto di vista di come approcciarsi a un progetto e su come doversi rapportare con il proprio team cercando sempre di spiegarsi il più chiaramente possibile.

Elena Lungu

Il mio contributo al progetto si è focalizzato sulla modellazione e implementazione del mazzo di carte (Deck e relative carte) e del portafoglio dei giocatori (Wallet) a livello di Model, oltre che sulla loro rappresentazione visiva all'interno della View. La parte più impegnativa è stata la gestione della logica delle carte e soprattutto delle carte speciali, e il corretto funzionamento del Wallet. Dal lato della View, la sfida principale è stata creare componenti grafiche (CardView e PlayerWalletView) che si aggiornassero dinamicamente e in modo pulito in risposta ai cambiamenti del Model. Ritengo il lavoro complessivo molto soddisfacente, con una netta separazione delle responsabilità che rispetta pienamente il pattern MVC. Questa esperienza è stata particolarmente formativa perché mi ha permesso di capire sul campo quanto sia fondamentale progettare interfacce (API) chiare per permettere agli altri membri del team di usare il mio codice senza doverne conoscere l'implementazione interna. In un'ottica di sviluppo futuro, il sistema del Wallet potrebbe essere ampliato introducendo meccaniche di prestiti o ricompense giornaliere, mentre l'interfaccia grafica delle carte trarrebbe enorme beneficio

dall'aggiunta di animazioni fluide per la distribuzione e lo scarto, rendendo l'esperienza di gioco molto più immersiva.

4.2 Difficoltà incontrate e commenti per i docenti

Evelyn Calandrini

Questo è stato il mio primo progetto e, soprattutto, la mia prima esperienza di sviluppo in un team. È stata un'esperienza molto formativa, che mi ha permesso di capire quanto sia importante non solo scrivere codice, ma anche progettare correttamente il software, comunicare in modo efficace con gli altri membri del gruppo e sviluppare componenti che possano integrarsi facilmente con il lavoro degli altri.

Giulia Bardi

Essendo il mio primo progetto inizialmente è stato difficile capire come coordinarsi e lavorare in team, però penso che mi sia servito molto per migliorare. Questo sia dal punto di vista puramente tecnica sia per comprendere l'importanza della comunicazione tra membri dello stesso gruppo.

Elena Lungu

Dal punto di vista tecnico, la vera sfida iniziale è stata imparare a utilizzare in maniera corretta Git: gestire i branch separati e sopravvivere ai conflitti di merge senza cancellare il lavoro altrui ha richiesto un po' di pratica. A livello architetturale, l'ostacolo maggiore è stato far dialogare le mie componenti con quelle del gruppo, mantenendo il disaccoppiamento delle parti nel rispetto del pattern MVC. Alla fine ho imparato l'importanza di Checkstyle e di un codice pulito per rendere la fase di integrazione decisamente più facile.

Appendice A

Guida utente



Figura A.1: Schermata principale del gioco

All'avvio dell'applicazione viene mostrato il menu principale, dal quale l'utente può scegliere se avviare una nuova partita, visualizzare le statistiche oppure uscire dall'applicazione. Da qualsiasi schermata successiva è sempre possibile tornare al menu principale premendo il pulsante Menù. Selezionando il pulsante Play, viene avviata una nuova partita. Una volta avviata la partita, la schermata di gioco mostra il tavolo completo: sono visibili il dealer e tutti i giocatori seduti, ciascuno con il rispettivo portafoglio. Per facilitare la comprensione del flusso di gioco:

- Il nome del giocatore il cui turno è attualmente in corso viene evidenziato.

- Accanto alle carte di ogni giocatore viene mostrato il punteggio attuale della mano, semplificando il calcolo durante i round con regole speciali.
- Al centro della schermata vengono costantemente aggiornati la fase di gioco corrente e il valore totale del piatto (pot).
- Nella parte in alto a destra viene indicato l'eventuale svolgimento di un round speciale.

Inoltre, durante la partita sono disponibili due pulsanti principali:

- Menu = che consente di tornare alla schermata principale;
- Pausa = che permette di sospendere la partita. Da questa schermata l'utente può scegliere se riprendere il round in corso, ricominciare il round dall'inizio oppure tornare direttamente al menu principale.

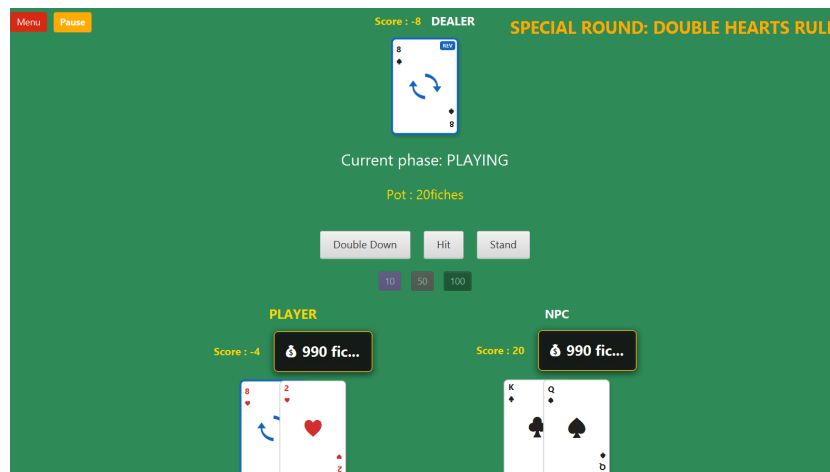


Figura A.2: Schermata di gioco in corso

Appendice B

Esercitazioni di laboratorio

evelyn.calandrini@studio.unibo.it

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p285016>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287284>